

CUSTOMIZE BEHAVIOR

Agent Skills in the SDK

Extend Claude with specialized capabilities using Agent Skills in the Claude Agent SDK

Overview

Agent Skills extend Claude with specialized capabilities that Claude autonomously invokes when relevant. Skills are packaged as `SKILL.md` files containing instructions, descriptions, and optional supporting resources.

For comprehensive information about Skills, including benefits, architecture, and authoring guidelines, see the [Agent Skills overview](#).

How Skills Work with the SDK

When using the Claude Agent SDK, Skills are:

1. **Defined as filesystem artifacts:** Created as `SKILL.md` files in specific directories (`.claude/skills/`)
2. **Loaded from filesystem:** Skills are loaded from filesystem locations governed by `settingSources` (TypeScript) or `setting_sources` (Python)
3. **Automatically discovered:** Once filesystem settings are loaded, Skill metadata is discovered at startup from user and project directories; full content loaded when triggered
4. **Model-invoked:** Claude autonomously chooses when to use them based on context
5. **Filtered via the `skills` option:** Discovered skills are enabled by default. Pass a list of skill names, `"all"`, or `[]` to control which are available in the session

Unlike subagents (which can be defined programmatically), Skills must be created as filesystem artifacts. The SDK does not provide a programmatic API for registering Skills.

❗ Skills are discovered through the filesystem setting sources. With default `query()` options, the SDK loads user and project sources, so skills in `~/.claude/skills/`, `<cwd>/.claude/skills/`, and `.claude/skills/` in any parent directory of `<cwd>` up to the repository root are available. If you set `settingSources` explicitly, include `'user'` or `'project'` to keep skill discovery, or use the [plugins option](#) to load skills from a specific path.

Using Skills with the SDK

Set the `skills` option on `query()` to control which Skills are available to the session. When omitted, discovered Skills are enabled and the Skill tool is available, matching CLI behavior. Pass `"all"` to enable every discovered Skill, a list of Skill names to enable only those, or `[]` to disable all. When you set `skills`, the SDK adds the Skill tool to `allowedTools` automatically. If you also pass an explicit `tools` list, include `"Skill"` in that list so Claude can invoke skills.

Once configured, Claude automatically discovers Skills from the filesystem and invokes them when relevant to the user's request.

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    options = ClaudeAgentOptions(
        cwd="/path/to/project", # Project with .claude/skills/
        setting_sources=["user", "project"], # Load Skills from filesystem
        skills="all", # Enable every discovered Skill
        allowed_tools=["Read", "Write", "Bash"],
    )

    async for message in query(
        prompt="Help me process this PDF document", options=options
    ):
        print(message)

asyncio.run(main())
```

To enable only specific Skills, pass their names. Names match the `name` field in `SKILL.md` or the Skill's directory name. Use `plugin:skill` for plugin-provided Skills.

```
options = ClaudeAgentOptions(skills=["pdf", "docx"])
```

The `skills` option is a context filter, not a sandbox. Unlisted Skills are hidden from the model and rejected by the Skill tool, but their files remain on disk and are reachable through Read and Bash.

Skill Locations

Skills are loaded from filesystem directories based on your `settingSources` / `setting_sources` configuration:

- **Project Skills** (`.claude/skills/`): Shared with your team via git - loaded when `setting_sources` includes `"project"`
- **User Skills** (`~/.claude/skills/`): Personal Skills across all projects - loaded when `setting_sources` includes `"user"`
- **Plugin Skills**: Bundled with installed Claude Code plugins

Creating Skills

Skills are defined as directories containing a `SKILL.md` file with YAML frontmatter and Markdown content. The `description` field determines when Claude invokes your Skill.

Example directory structure:

```
.claude/skills/processing-pdfs/  
└── SKILL.md
```

For complete guidance on creating Skills, including SKILL.md structure, multi-file Skills, and examples, see:

- **Agent Skills in Claude Code**: Complete guide with examples
- **Agent Skills Best Practices**: Authoring guidelines and naming conventions

Tool Restrictions

- ❗ The `allowed-tools` frontmatter field in SKILL.md is only supported when using Claude Code CLI directly. It **does not apply when using Skills through the SDK**.

When using the SDK, control tool access through the main `allowedTools` option in your query configuration.

To control tool access for Skills in SDK applications, use `allowedTools` to pre-approve specific tools. Without a `canUseTool` callback, anything not in the list is denied:

- ❗ Import statements from the first example are assumed in the following code snippets.

```
options = ClaudeAgentOptions(  
    setting_sources=["user", "project"], # Load Skills from filesystem  
    skills="all",  
    allowed_tools=["Read", "Grep", "Glob"],  
)  
  
async for message in query(prompt="Analyze the codebase structure",  
    options=options):  
    print(message)
```

Discovering Available Skills

To see which Skills are available in your SDK application, simply ask Claude:

```
options = ClaudeAgentOptions(  
    setting_sources=["user", "project"], # Load Skills from filesystem  
    skills="all",  
)  
  
async for message in query(prompt="What Skills are available?",  
    options=options):  
    print(message)
```

Claude will list the available Skills based on your current working directory and installed plugins.

Testing Skills

Test Skills by asking questions that match their descriptions:

```
options = ClaudeAgentOptions(  
    cwd="/path/to/project",  
    setting_sources=["user", "project"], # Load Skills from filesystem  
    skills="all",  
    allowed_tools=["Read", "Bash"],  
)  
  
async for message in query(prompt="Extract text from invoice.pdf",  
    options=options):  
    print(message)
```

Claude automatically invokes the relevant Skill if the description matches your request.

Troubleshooting

Skills Not Found

Check settingSources configuration: Skills are discovered through the `user` and `project` setting sources. If you set `settingSources` / `setting_sources` explicitly and omit those sources, skills are not loaded:

```
# Skills not loaded: setting_sources excludes user and project  
options = ClaudeAgentOptions(setting_sources=[], skills="all")  
  
# Skills loaded: user and project sources included  
options = ClaudeAgentOptions(  
    setting_sources=["user", "project"],  
    skills="all",  
)
```

For more details on `settingSources` / `setting_sources`, see the [TypeScript SDK reference](#) or [Python SDK reference](#).

Check working directory: The SDK loads Skills from `.claude/skills/` in the `cwd` option and in every parent directory up to the repository root. Ensure `cwd` points at or below the directory

containing `.claude/skills/` , within the same repository:

```
# Ensure your cwd points to the directory containing .claude/skills/
options = ClaudeAgentOptions(
    cwd="/path/to/project", # .claude/skills/ here or in a parent directory
    setting_sources=["user", "project"], # Loads skills from these sources
    skills="all",
)
```

See the “Using Skills with the SDK” section above for the complete pattern.

Verify filesystem location:

```
# Check project Skills
ls .claude/skills/*/SKILL.md

# Check personal Skills
ls ~/.claude/skills/*/SKILL.md
```

Skill Not Being Used

Check the `skills` option: If you passed a `skills` list, confirm the skill’s name is included. Passing `[]` disables all skills.

Check the description: Ensure it’s specific and includes relevant keywords. See [Agent Skills Best Practices](#) for guidance on writing effective descriptions.

Additional Troubleshooting

For general Skills troubleshooting (YAML syntax, debugging, etc.), see the [Claude Code Skills troubleshooting section](#).

Related Documentation

Skills Guides

- [Agent Skills in Claude Code](#): Complete Skills guide with creation, examples, and troubleshooting
- [Agent Skills Overview](#): Conceptual overview, benefits, and architecture

- **Agent Skills Best Practices**: Authoring guidelines for effective Skills
- **Agent Skills Cookbook**: Example Skills and templates

SDK Resources

- **Subagents in the SDK**: Similar filesystem-based agents with programmatic options
- **Slash Commands in the SDK**: User-invoked commands
- **SDK Overview**: General SDK concepts
- **TypeScript SDK Reference**: Complete API documentation
- **Python SDK Reference**: Complete API documentation